CLASE 4 · MÓDULO 3 (PARTE 2)

Entrenar una neurona

Regla delta, entrenamiento completo, ADALINE y redes de varias salidas

github.com/datahuba/curso-redes-neuronales · Basado en el curso del Dr. J. Irving Vasquez (jivg.org)

Agenda de hoy

Tiempo	Tema
0:00	Repaso de la Clase 3
0:10	Medir el error
0:25	La regla delta
0:50	Práctica: un paso del descenso
1:10	Descanso
1:20	Entrenamiento completo e inicialización
1:35	Práctica: entrenar una neurona
2:00	ADALINE
2:08	Red con varias salidas
2:20	Quiz y cierre

Al terminar la clase podrás…

1

Medir

el error de una red con el MSE.

2

Aplicar

un paso de la regla delta a mano.

3

Entrenar

una neurona completa y medir su exactitud.

4

Inicializar

los pesos al azar con Xavier.

5

Calcular

una capa con varias salidas.

Repaso de la Clase 3

¿Sigmoide?

Convierte h en un número entre 0 y 1, suave.

¿Derivada?

La pendiente: hacia dónde sube.

¿Descenso?

$x \leftarrow x - \eta \cdot f'(x)$

Hoy: bajar por el error de la red moviendo los pesos. Eso es entrenar.

Medir el error

Medida	Fórmula
Error residual	$e = y - \hat{y}$
SSE	$\Sigma\ (y - \hat{y})^2$
MSE (promedio)	$(1/m)\ \Sigma\ (y - \hat{y})^2$

¿Por qué al cuadrado? +1 y −1 no se cancelan, y los errores grandes pesan más.

Ejemplo

`y = [1, 0, 1]`

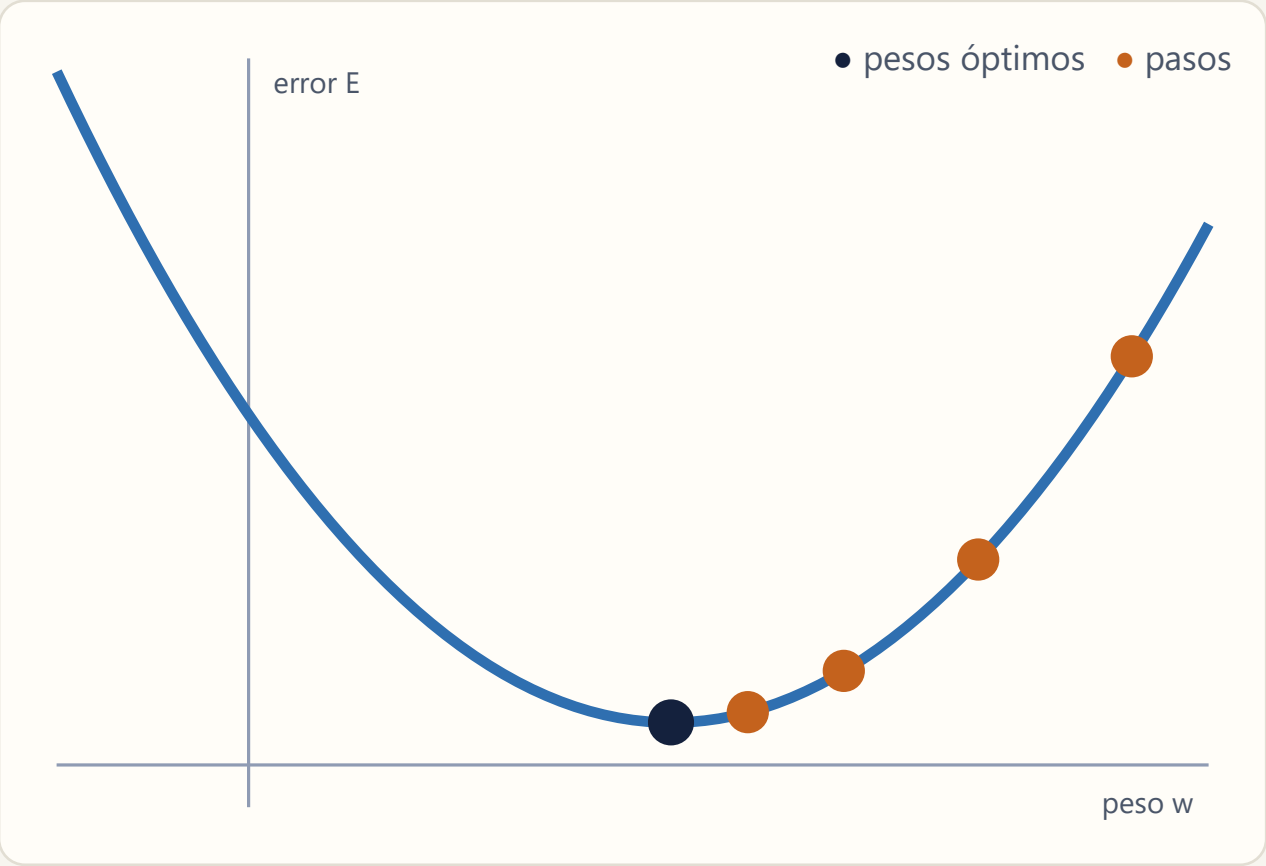
`ŷ = [0.8, 0.3, 0.6]`

errores²: 0.04, 0.09, 0.16

SSE = 0.29

MSE = 0.29 / 3 ≈ **0.0967**

La pérdida es una montaña



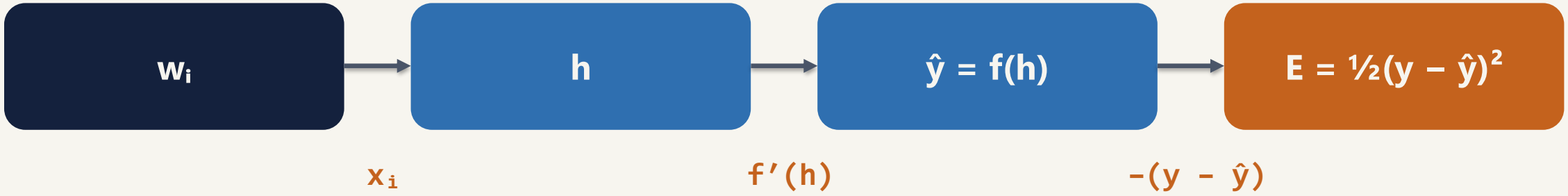
Entrenar = bajar al valle del error moviendo los pesos.

Es lo mismo de la clase 3:

x → **peso w**

f(x) → **error E**

¿Cuánto cambia el error si muevo un peso?



Regla de la cadena: se multiplican los efectos de cada eslabón

$$\partial E/\partial w_i = -(y - \hat{y}) \cdot f'(h) \cdot x_i$$

$$w \leftarrow w - \eta \cdot \partial E/\partial w \quad \Rightarrow \quad w_i \leftarrow w_i + \eta\,(y - \hat{y})\, f'(h)\, x_i$$

La regla delta en tres pasos

1 · Término de error

$$\delta = (y - \hat{y}) \cdot f'(h)$$

Cuánto y hacia dónde corregir

2 · Incremento

$$\Delta w_i = \eta \cdot \delta \cdot x_i$$

$\Delta b = \eta \cdot \delta$

3 · Actualización

$$w_i \leftarrow w_i + \Delta w_i$$

$b \leftarrow b + \Delta b$

Con la sigmoide: $f'(h) = \hat{y} \cdot (1 - \hat{y})$

Ejemplo: un paso de la regla delta

`x = [1, 1] · y = 1 · w = [0.1, 0.2] · b = 0 · η = 2 · sigmoide`

Paso	Cálculo
Predicción	$h = 0.3 \rightarrow \hat{y} = \sigma(0.3) \approx 0.5744$
Derivada	$f'(h) = 0.5744 \cdot 0.4256 \approx 0.2445$
Término de error	$\delta = (1 - 0.5744) \cdot 0.2445 \approx 0.1040$
Incremento	$\Delta w = 2 \cdot 0.1040 \cdot [1, 1] \approx [0.2081, 0.2081]$ · $\Delta b \approx 0.2081$
Actualización	$w \approx [0.3081, 0.4081]$ · $b \approx 0.2081$
Comprobar	$\hat{y} \approx 0.7159$ · pérdida 0.0905 → 0.0404 ✓

Perceptrón vs. regla delta

	Perceptrón (Clase 2)	Regla delta (hoy)
Activación	Escalón	Continua (sigmoide, lineal…)
Corrección	$\eta \cdot (y - \hat{y}) \cdot x$	$\eta \cdot (y - \hat{y}) \cdot \mathbf{f'(h)} \cdot x$
De dónde sale	Intuición de Rosenblatt	Descenso por gradiente
El error indica	Solo si acertó	Qué tan lejos estuvo

¡Casi la misma fórmula! La diferencia es la pendiente de la activación.

NOTEBOOKS · 20 MIN · 3 PUNTOS

Práctica · 1_06_descenso_unpaso

TODO 1 pt error_term: $\delta = (y - \hat{y}) \cdot f'(h)$

TODO 1 pt incremento: $\eta \cdot \delta \cdot X$

TODO 1 pt actualizacion: sumar a W y b

Pistas: h con neurona.combinacion_lineal(x), ŷ con neurona.forward(x), f′ con sigmoid_prime(h).

```
# comprobaciones
Salida: 0.5744
Término de error: 0.1040
Incremento: [0.2081 0.2081]
Pesos actualizados: [0.3081 0.4081]
```

Al final: vuelve a predecir. Debe dar ≈ 0.7159, más cerca de y = 1.

Descanso · 10 min

Después: entrenar con todos los datos

El algoritmo completo

1. Inicializar los pesos al azar (Xavier).
2. Para cada época: incrementos acumulados = 0.
3. Para cada ejemplo: $\hat{y}$, δ y sumar $\delta \cdot x$ al acumulado.
4. Actualizar: $w \mathrel{+}= \eta \cdot \text{acumulado} / n_\text{ejemplos}$.
5. Medir el error (MSE) y repetir.
6. Al final: exactitud con datos de prueba.

Vocabulario

Época: una pasada por todos los datos.

Por lotes: una actualización por época con el promedio.

Prueba: datos que no se usaron para entrenar.

Inicialización de los pesos

¿Por qué no en cero?

Todas las salidas serían iguales: hay que romper la simetría.

¿Por qué pequeños?

Con pesos grandes la sigmoide se satura y aprende lento.

Xavier

$w \sim U(-1/\sqrt{n}, +1/\sqrt{n})$

n = número de entradas

Ejemplo: con n = 6 entradas, cada peso se elige al azar entre **−0.408 y +0.408**

Los datos de la práctica: admisiones

Columna	Qué es
gre	Puntaje del examen GRE (escalado)
gpa	Promedio (escalado)
rank_1 … rank_4	Prestigio de la universidad de origen (0/1)
admit	Salida: 1 admitido, 0 no

400 alumnos · 6 entradas

90 % entrenamiento (360) · 10 % prueba (40)

Exactitud

`aciertos / predicciones`

La red dice “admitido” si $\hat{y} > 0.5$.

Meta del notebook: **más de 0.74**

NOTEBOOKS · 25 MIN · 3 PUNTOS

Práctica · 1_07_descenso_por_gradiente

TODO 1 pt pesos con Xavier (np.random.uniform)

TODO 1 pt completar el ciclo de entrenamiento

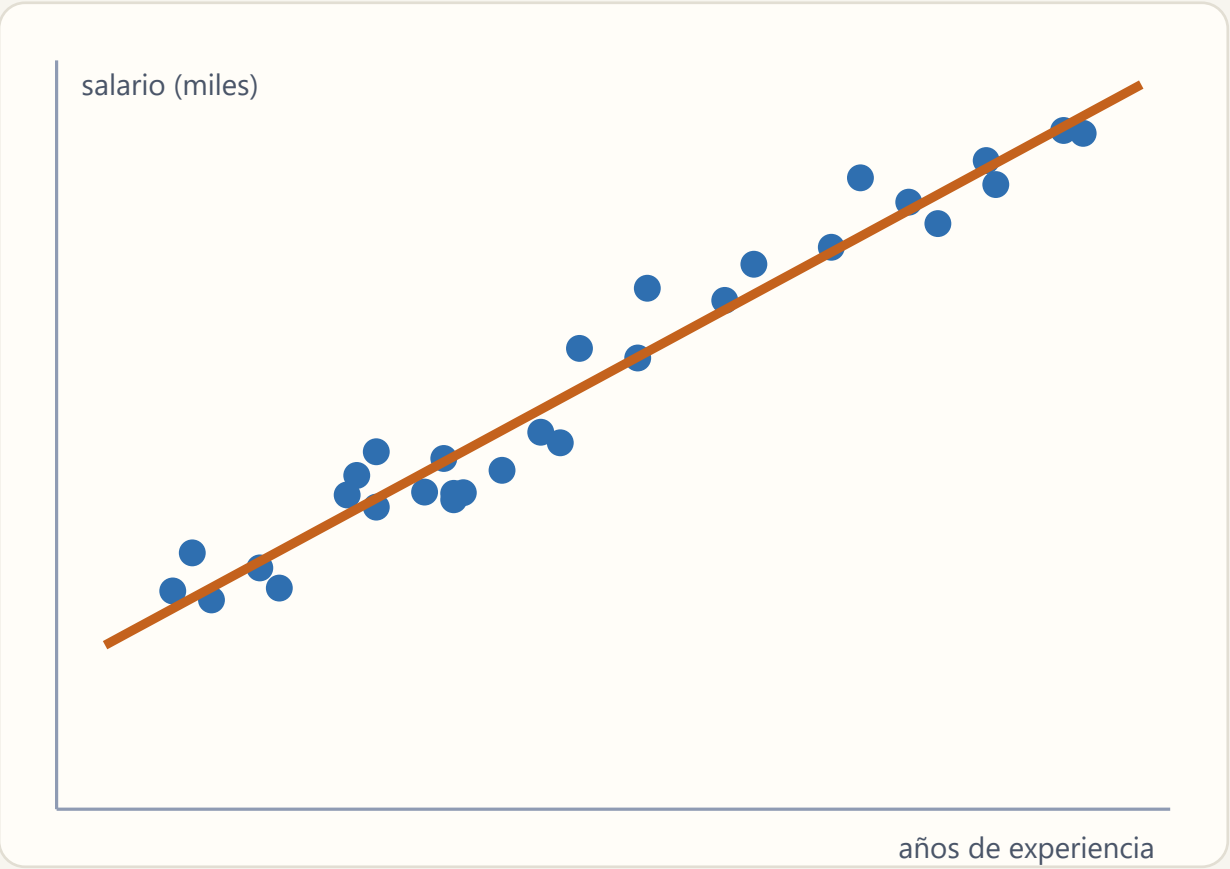
1 pt épocas hasta exactitud > 0.74

Pista: sigue los pasos 3 y 4 del algoritmo (diapositiva 13).

```
# referencia (semilla 42, learnrate 0.5)
# épocas  pérdida  exactitud
#   10     0.225    0.60
#  100     0.205    0.775
# 1000     0.197    0.725
```

Usa `weights += …` y no weights = weights + …: la neurona comparte el arreglo; si creas uno nuevo, la pérdida no baja.

ADALINE: regresión con una neurona



Activación lineal: $\hat{y} = w \cdot x + b$

Derivada: $f'(h) = 1$

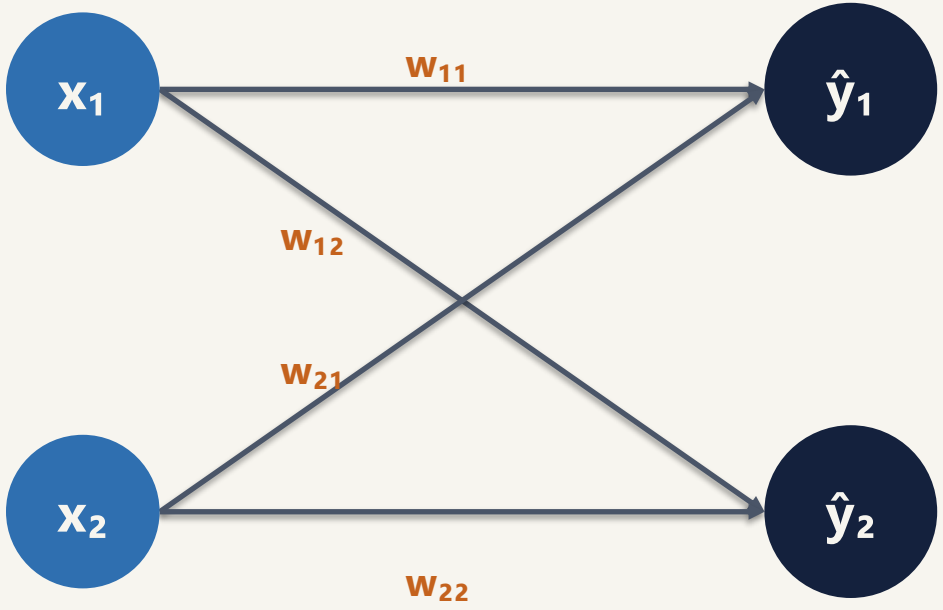
Regla: $w \leftarrow w + \eta(y - \hat{y})x$

salario ≈ 9 450 · años + 24 848

5 años → ≈ 72 098

Lección: escalar los datos acelera mucho el entrenamiento.

Red de una capa con varias salidas



$$H = X \cdot W + B \qquad \hat{Y} = f(H)$$

W: matriz de m entradas × n salidas.

B: un sesgo por salida.

f: se aplica a cada elemento de H.

Parámetros: $(m + 1) \cdot n$

Cada entrada se conecta con cada salida: “completamente conectada”.

Ejemplo: 2 entradas, 2 salidas

`X = [8, 4] · W = [[−0.1, 0.6], [−0.8, 0.3]] · B = [−0.7, 0.3] · sigmoide`

Salida	Combinación lineal	Sigmoide
$\hat{y}_1$	$8 \cdot (-0.1) + 4 \cdot (-0.8) - 0.7 = -4.7$	$\sigma(-4.7) \approx 0.0090$
$\hat{y}_2$	$8 \cdot 0.6 + 4 \cdot 0.3 + 0.3 = 6.3$	$\sigma(6.3) \approx 0.9982$

Parámetros: `(2 + 1) · 2 = 6`

Reto: el libro dice H = [−7.7, 6.3]. ¿Dónde está el error?

Quiz · 5 minutos

1. ¿Por qué el error se eleva al cuadrado?
2. ¿Cuáles son los tres pasos de la regla delta?
3. ¿Qué añade la regla delta respecto a la del perceptrón?
4. ¿Por qué no se inicializan los pesos en cero?
5. ¿Cuántos parámetros tiene una capa con 3 entradas y 2 salidas?

Para llevar

1. Entrenar = bajar por el error moviendo los pesos.
2. Regla delta: $\delta = (y - \hat{y}) \cdot f'(h)$, $\Delta w = \eta \cdot \delta \cdot x$.
3. Una sola capa sigue sin resolver XOR.

Próximo módulo: redes multicapa y retropropagación.

Tarea: notebooks 1_06 y 1_07 (6 pts)

Leer: libro, “Redes neuronales multicapa”